



Three-Day Take-Home Assignment for Junior Machine Learning Engineer

Submission Deadline: End of day (EOD) Pacific Standard Time (PST) on Thursday of this week. Please ensure all deliverables are submitted by this time — late submissions will not be reviewed.

Introduction

Welcome, and thank you for your interest in the **Junior Machine Learning Engineer** position! This take-home assignment is designed to evaluate your practical skills in data acquisition, applied NLP, agent design, and communication of results. You will have **3 days (72 hours)** from the time you receive this assignment to complete and submit your work.

You are building a prototype. It does not need to be super clean or production-grade — a working, understandable proof of concept is the goal. It only needs to **run locally on your machine**; it does **not** need to be deployed anywhere. The assignment is **open-ended**: we want to see **how you think** — how you investigate an unfamiliar data source, justify decisions, handle trade-offs, and communicate findings. A thoughtful, well-documented solution with a simple detection model will always score higher than a complex model with no explanation. That said, this assignment is deliberately demanding — expect to make and defend non-trivial technical decisions.

Estimated effort: 12–16 hours of focused work spread across the 3 days. Plan your time carefully — completing every required component within the window is part of the challenge.

The Problem: Truth Social Stock-Mention Alert Agent

You will design and build an **autonomous agent** that monitors the Truth Social account of **Donald Trump (@realDonaldTrump)** and **sends an alert/update whenever he posts about a stock** — whether by ticker symbol (e.g., \$DJT, TSLA) or company name (e.g., “Tesla”, “Boeing”, “Trump Media”).

The Data Source

- **Source:** Truth Social, the public account @realDonaldTrump
- **Constraint:** Truth Social does **not** offer public API access. **Figuring out how to reliably read the account’s posts without official API access is a core part of this assignment.** Investigate your options — e.g., inspecting the site’s network traffic for unofficial endpoints, HTML scraping, third-party mirrors, or RSS-style proxies — and justify the approach you choose in terms of reliability, latency, and fragility.
- The posts are publicly visible; you do not need an account. Be a good citizen: respect reasonable polling intervals and rate limits, and document any terms-of-service considerations your approach raises.
- **Testing tip:** To prove your pipeline works end-to-end, you are welcome to point your agent at a **dummy/test account or profile you control** and post sample content there. Demonstrating the prototype against a test account is perfectly acceptable.

Assignment Tasks

The assignment is divided into four parts. A suggested day-by-day schedule is provided below, but you are free to organize your time however you prefer — the scope is open-ended, so prioritize what you find most interesting and important.

Part 1: Data Acquisition Without an API

Build the ingestion layer of your agent:

- Reverse-engineer or otherwise devise a method to **read new posts from the account without official API access**. Document at least **two candidate approaches** you considered and why you chose yours.
- Implement **polling or streaming** logic with a justified refresh interval — analyze the trade-off between alert latency and request volume.
- Handle **deduplication** (never process or alert on the same post twice), including across agent restarts (persist state)
- Handle **failure modes**: network errors, layout/endpoint changes, rate limiting, and malformed content. Your agent should degrade gracefully and log clearly.
- Collect a **historical sample of posts** (as many as your method allows) to use for developing and evaluating your detection logic in Part 2.

Part 2: Stock-Mention Detection

- Build a component that classifies each post as **stock-related or not**, and extracts **which stock(s)**
- Go beyond naive keyword matching. Handle at least:
 - **Ticker symbols** with and without \$ prefixes (and the ambiguity of short all-caps words like “A”, “IT”, “DJT”)
 - **Company names and colloquial references** (e.g., “Truth Social” → DJT, “the electric car company”)
 - **False positives** — e.g., “Apple” the fruit vs. Apple Inc., political uses of “stock”.
- Include a **machine learning component** (e.g., an NER model, embedding-based entity linking, a fine-tuned or zero-shot classifier)
- Build a **labeled evaluation set** from your collected historical posts (hand-label at least 100 posts)
- Perform an **error analysis**: characterize the posts your detector misclassifies. Are there identifiable patterns where it systematically fails?

Part 3: The Alerting Agent

- Wire the pieces into a **continuously running agent** that, upon detecting a stock-related post, **sends an update** via at least one channel (e.g., email, Slack/Discord webhook, Telegram bot, or SMS via a free tier)
- Each alert must include: the post text, timestamp, detected ticker(s)/company name(s)
- Address **reliability concerns**: retries on delivery failure, idempotency (no duplicate alerts)
- Measure and report your agent’s **end-to-end latency** (post published → alert delivered)
- Design (implement if time allows) at least one **monitoring signal** that would tell you the ingestion method has silently broken (e.g., “no posts seen in N hours” heartbeat).

Part 4: Write-Up & Recommendations

Include a short report (README.md or a final notebook section, **800–1,200 words**) covering:

1. **Summary of approach** — what you did and why, especially your no-API read strategy and its trade-offs.
2. **Results** — detection metrics (rule-based vs. ML), labeled-set methodology, and measured alert latency.
3. **Robustness & ethics** — how fragile is your ingestion method, how would you detect and recover from breakage, and what legal/ToS/rate-limiting considerations apply to reading a site without API access?

4. **Limitations & next steps** — what you would do with more time, including at least one concrete idea for scaling this to monitor **multiple accounts** and one for evaluating detection quality **in production** without hand-labeling everything.

Suggested 3-Day Schedule

Day	Focus	Suggested Activities
1	Crack the data source	Investigate read strategies, build ingestion + dedup, collect historical posts
2	Detect stock mentions	Label evaluation set, rule-based baseline, ML detector, metrics, error analysis
3	Ship the agent	Alert delivery, reliability handling, latency measurement, monitoring, write-up, tests, code cleanup

Deliverables

Submit a **GitHub repository** (or a .zip archive) containing:

1. **Code** — the agent source, plus any notebooks used for detector development. Notebooks should run top-to-bottom without errors. Remember: this is a prototype — it doesn't need to be super clean, just understandable and runnable.
2. **README.md** — setup instructions, how to run the agent locally, and your write-up (or a link to it).
3. **requirements.txt** or **environment.yml** — so we can reproduce your environment.
4. **Runnable agent entry point** (**agent.py** or similar) — starts the monitor loop and sends real alerts to a configurable channel. **This is required, not optional.** Running locally is sufficient — no deployment needed.
5. **Labeled evaluation set** — your hand-labeled posts and the script that computes your reported metrics.
6. **Unit tests** — at least a handful of tests covering your parsing/detection logic and dedup/alerting behavior (e.g., with **pytest**), using recorded fixtures so tests run offline.
7. *(Optional bonus)* A **Dockerfile** for reproducible deployment, a simple dashboard of detected mentions, and/or sentiment scoring on stock-related posts (bullish/bearish).

Submission Instructions

- Email your repository link or archive to me within **72 hours** of receiving this assignment.
- Use a **fixed random seed** wherever applicable so results are reproducible.
- Include a short screen recording or log excerpt demonstrating a **live end-to-end alert** if you achieved one — a demo against a dummy/test account you control counts.

Evaluation Criteria

Your submission will be scored against the following rubric:

Criterion	Weight	What We Look For
	30%	



Criterion	Weight	What We Look For
Data acquisition & agent engineering		A working no-API read strategy, sound polling/dedup/state design, graceful failure handling, measured latency
Detection methodology	25%	Rigorous baseline-vs-ML comparison, honest labeled evaluation, sensible handling of ticker/name ambiguity
Code quality & reproducibility	20%	Understandable, organized, runnable prototype code (polish not required); offline-capable tests; pinned dependencies
Communication	20%	Clear write-up, justified decisions, thoughtful robustness and ethics discussion
Bonus	5%	Dockerfile, dashboard, sentiment scoring, multi-account design, or other extras

Important: Depth beats breadth. We would rather see one ingestion method engineered robustly than three methods duct-taped together carelessly.

Ground Rules

- **Allowed:** Standard open-source libraries (selenium, requests, beautifulsoup4, playwright, spacy, transformers, scikit-learn, etc.)
- **Not allowed:** Submitting copied end-to-end scraper or bot solutions from tutorials without attribution or understanding;

Questions?

If anything is unclear or you hit a blocker (e.g., ingestion dead-ends), email me. Asking good clarifying questions is a positive signal, not a negative one.

Conclusion

This assignment mirrors the day-to-day work of our ML team: getting data from uncooperative real-world sources, building sensible detection models, and shipping agents that people can rely on. Remember — this is an open-ended prototype that runs locally, not a polished deployed product. Focus on **clarity, correctness, and communication** rather than chasing the flashiest solution. We’re excited to see your approach — good luck, and enjoy the challenge!